

操作系统课程设计报告：基于多线程的十字路口交通控制系统

项目名称：交通控制模拟系统 (Traffic Control Simulation)

开发环境：Python 3.14.4 (Tkinter, threading)

姓名：张昊

学号：2452770

日期：2026年5月3日

一、项目概述

本系统旨在模拟十字路口的交通控制情况，通过多线程技术实现车辆生成、红绿灯控制及车辆通行调度。系统严格遵循“操作系统处理机管理”的核心思想，利用线程模拟并发任务，使用信号量与互斥锁解决资源竞争问题，实现了普通车辆按红绿灯通行、特权车辆（消防车、救护车、警车）无视红灯优先通行的逻辑。

二、系统设计方案 (紧贴考核点)

本报告将围绕项目要求的四个核心考核点进行详细阐述：

1. 实现 (Implementation) —— 多线程架构与任务划分

根据项目需求，系统初始化创建5个核心任务（线程），并动态生成车辆任务。

• 任务划分与优先级：

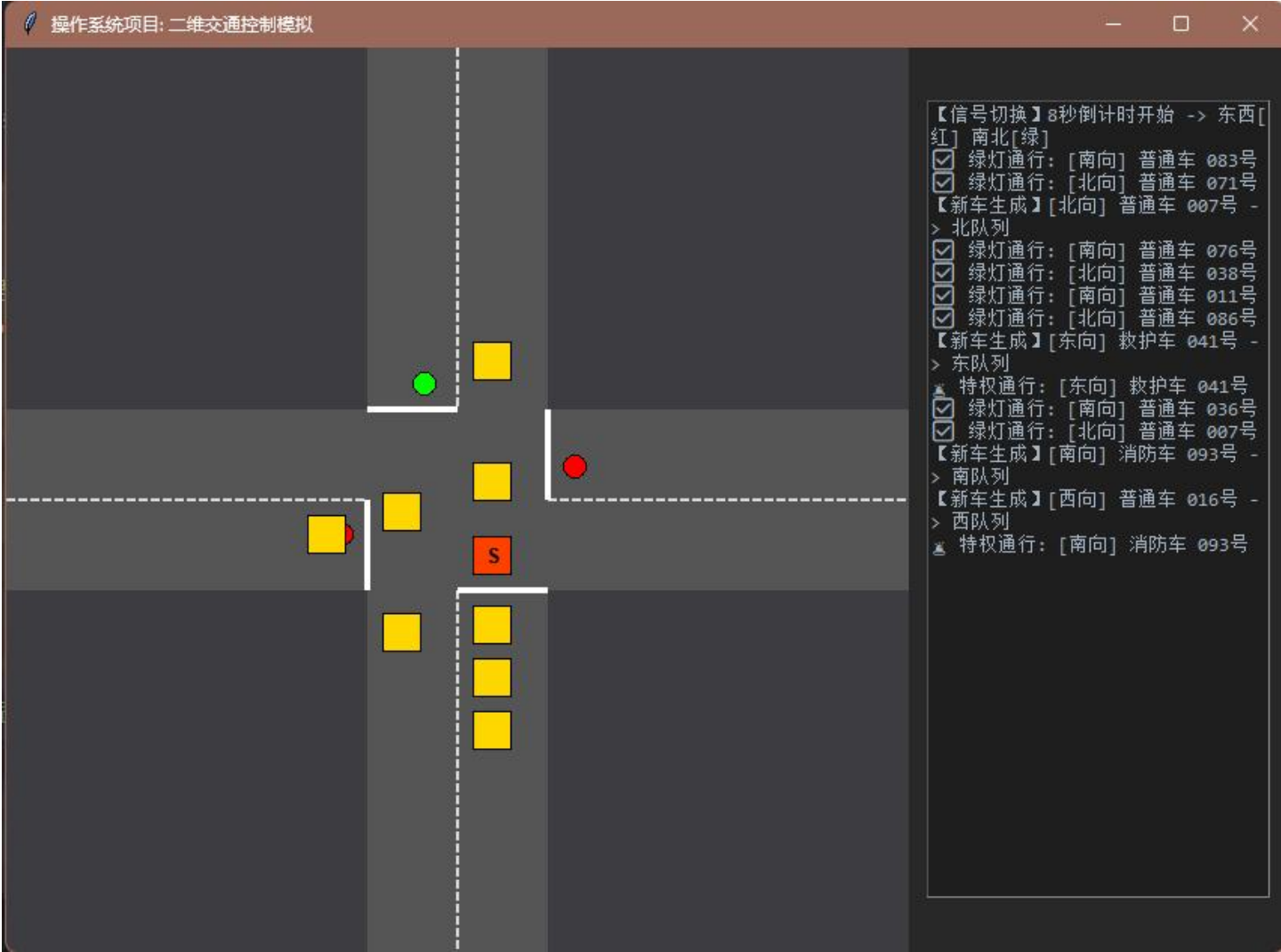
- **任务 1 (ControllerThread)：总控任务。**负责红绿灯的周期性切换（8秒倒计时）以及启动车辆生成器。该任务优先级最高（通过守护线程与锁机制保证）。
- **任务 2~5 (DirectionThread)：方向调度任务。**分别为东、西、南、北四个方向创建。这些任务优先级相同，负责检查本方向队列并调度车辆通行。
- **动态任务 (Vehicle Thread)：**每辆生成的车（普通/特权）在启动通行时会创建一个独立的生命周期线程（`start_cross_thread`），用于模拟车辆通过路口的时间，避免阻塞调度线程。

• 代码实现逻辑：

- **总控线程 (`traffic_logic.py`)：**使用 `global ew_green` 变量控制灯色，并通过 `time.sleep(8)` 模拟8秒周期。
- **方向线程 (`traffic_logic.py`)：**使用 `while True` 循环不断轮询队列。特权车通过非阻塞方式直接放行，普通车需判断 `ew_green` 状态。

2. 界面 (Interface) —— 二维可视化与实时监控

系统采用 Python Tkinter 实现二维图形化界面，分为“模拟区”与“日志区”两部分。



- 模拟区 (Canvas):
 - 静态元素: 绘制十字路口、中心虚线、停止线 (白线)。
 - 动态元素:
 - 红绿灯: 根据 `ew_green` 变量实时切换颜色 (东西向绿/南北向红, 反之亦然)。
 - 车辆显示: 使用不同颜色的方块代表不同车辆 (金黄=普通, 橘红=消防, 亮白=救护, 亮蓝=警车)。特权车标有 "S" 字样。
 - 动画原理: GUI 主线程以 50ms (20FPS) 刷新率调用 `update_gui`, 根据车辆的 `start_cross_time` 计算进度 (`progress`) 并重绘位置。
- 日志区 (Text Widget):
 - 实时输出系统日志, 包括红绿灯切换、新车生成、特权车通行、绿灯放行等事件, 便于调试与观察调度逻辑。

3. 算法 (Algorithm) —— 调度策略与同步机制

这是本项目的核心, 重点考察对信号量和互斥的理解。

- 数据结构设计:

- 使用 `queue.Queue` (先进先出) 存储每个方向的车辆。Queue 内部自带锁机制，保证了多线程入队的安全性。
- 每个方向维护两个队列：`ordinary` (普通) 和 `special` (特权)，确保特权车优先调度。
- **调度算法 (Scheduling Logic):**
 - **特权优先算法:** 在 `DirectionThread` 中，线程首先检查 `special` 队列是否为空。若不为空，直接取出车辆并启动其独立线程通行，无视红灯状态。
 - **红绿灯算法:** 对于普通车，线程检查全局变量 `ew_green`。如果是东西向绿灯且当前方向为东西，或者南北绿灯且当前方向为南北，则放行。
 - **非阻塞通行:** 车辆启动后，通过 `threading.Thread(target=v.start_cross_thread)` 在独立线程中执行 `time.sleep(speed)`。这样调度线程不会被“卡住”，可以立即处理下一辆车，模拟了真实的车流并发。
- **同步与互斥机制:**
 - **互斥锁 (`threading.Lock`):**
 - **灯锁 (`light_lock`):** 在 `ControllerThread` 切换 `ew_green` 状态时，以及在 `DirectionThread` 读取该状态时加锁，防止读写冲突。
 - **临界资源保护:**
 - 全局列表 `active_vehicles` (正在通行的车) 被 GUI 线程读取，被车辆线程修改。虽然本实现利用了 Python GIL 的短期一致性，但在严谨设计中应使用锁保护（项目文档中已提及此注意事项）。

4. 可行性 (Feasibility) —— 逻辑验证与性能分析

- **需求满足度验证:**
 - ✓ **多线程概念:** 系统包含 1 个 GUI 主线程，1 个总控线程，4 个方向线程，N 个车辆线程，完美体现了多线程并发。
 - ✓ **调度机制:** 实现了基于优先级的抢占式调度（特权车）与基于时间片的轮转调度（红绿灯）。
 - ✓ **信号量/同步:** 使用了 `Lock` 保护共享变量，使用 `Queue` 作为线程安全的缓冲区。
 - ✓ **车辆行为:**
 - **速度差异:** 代码中 `Vehicle` 类定义 `speed=0.5` (特权) VS `speed=1.5` (普通)，实现了特权车速度快于普通车。
 - **闯红灯:** `DirectionThread` 逻辑中，特权车判断不依赖 `ew_green`，实现了红灯直行。
- **性能与扩展性:**
 - **优点:** 架构清晰，职责分离。将车辆移动逻辑放入独立线程，避免了“一辆车过马路阻塞整个队列”的逻辑错误。
 - **局限性:** 随着车辆数量激增，线程数量可能过多（Python GIL 限制）。但在教学演示规模下（几十辆车），系统运行流畅，完全满足演示需求。

三、代码关键片段解析

为了支撑上述设计方案，以下是核心代码逻辑的解析：

1. 互斥锁保护灯状态 (traffic_logic.py)

```
light_lock = threading.Lock() # 全局锁

# 在 ControllerThread 中切换灯
with light_lock:
    ew_green = not ew_green

# 在 DirectionThread 中读取灯
with light_lock:
    if (is_ew and ew_green) or (not is_ew and not ew_green):
        can_pass = True
```

解析：确保灯状态切换时，调度线程不会读取到中间状态。

2. 车辆独立线程实现非阻塞 (models.py)

```
def start_cross_thread(self, active_list):
    active_list.append(self) # 加入活跃列表（GUI绘制）
    time.sleep(self.speed)   # 模拟通过时间（不阻塞调度线程）
    active_list.remove(self) # 通行结束移除
```

解析：这是实现“多车并发”的关键。调度线程只负责“发车”（启动线程），不负责“等车过完”。

3. 特权车优先调度 (traffic_logic.py)

```
# 优先检查特权队列
if not queues[direction]["special"].empty():
    vehicle = queues[direction]["special"].get()
    # 启动特权车线程...

# 普通车需检查红绿灯
elif not queues[direction]["ordinary"].empty() and can_pass:
    # 启动普通车线程...
```

四、总结

本系统成功实现了操作系统课程中关于**处理机管理**的教学目标。通过 Python 的多线程模块，我们将抽象的“进程调度”、“信号量”概念具象化为可视化的交通控制系统。

- **创新点：**采用了“车辆生命周期独立线程”的设计，解决了传统单线程模拟中“阻塞导致无法并发”的难题。
- **不足与改进：**目前的 `active_vehicles` 列表访问未加锁，在极端高并发下可能报错（`List index out of range`），未来可引入 `threading.RLock` 或使用线程安全的队列进行优化。